■ TEATRO: “REACT EN PROBLEMAS”

Producto creativo sobre React: Componentes, JSX y TSX

■ Integrantes

• Integrante 1: REACT ■ – el líder del equipo
• Integrante 2: COMPONENTES ■ – el trabajador que quiere hacerlo todo
• Integrante 3: JSX / TSX ■■■ – personaje creativo y luego policía del código

■ Escena 1: El problema

REACT: ¡Buenos días, equipo! Tenemos una misión muy importante: ¡crear una aplicación moderna!

COMPONENTES: ¡Yo puedo hacerlo todo!

REACT: ¿Todo?

COMPONENTES: ¡Sí! Puedo hacer el menú, los botones, las tarjetas, el formulario, la página...

JSX/TSX: ¡Oye, tranquilo! ¿Quieres hacer también el café?

COMPONENTES: Si me dan código para hacerlo... ¡sí!

JSX/TSX: Eso explica por qué tu código siempre termina con 500 errores.

COMPONENTES: ¡No son errores! Son... características sorpresa. ■

■ Escena 2: Los componentes

REACT: Primero necesitamos hablar de los **componentes**.

COMPONENTES: ¡Ese es mi momento!

REACT: Un componente es una parte reutilizable de una aplicación React.

COMPONENTES: Exactamente. Puedo ser un botón, un menú, una tarjeta o incluso una parte completa de una página.

JSX/TSX: ¿Y si necesitamos diez botones?

COMPONENTES: ¡Me copian diez veces!

REACT: Justamente. El componente se puede **reutilizar**.

COMPONENTES: ¡Por fin una tecnología que respeta mi descanso! ■

■ Escena 3: JSX

REACT: Ahora necesitamos construir la interfaz.

JSX/TSX: ¡Llamaron al experto! Soy **JSX**, una sintaxis que permite escribir una estructura parecida a HTML dentro de JavaScript.

COMPONENTES: ¿Entonces eres HTML?

JSX/TSX: ¡No exactamente! JSX permite combinar la estructura visual con la lógica de JavaScript.

JSX/TSX: Ejemplo: `function Saludo() { return <h1>¡Hola, mundo!</h1>; }`

COMPONENTES: ¡Ah! Puedo escribir algo parecido a HTML dentro de JavaScript.

JSX/TSX: ¡Exactamente!

COMPONENTES: Entonces eres como un traductor.

JSX/TSX: Más o menos. ■

■ Escena 4: TSX

REACT: Todo está funcionando... pero necesitamos algo más seguro.

JSX/TSX: ¡Me toca otra vez! Ahora soy **TSX**, la combinación de TypeScript + JSX.

COMPONENTES: ¿Y qué tienes de diferente?

JSX/TSX: Además de permitir JSX, utilizo las ventajas de TypeScript, como los tipos de datos.

JSX/TSX: Por ejemplo, puedo indicar que un nombre debe ser un texto: `nombre: string`.

COMPONENTES: ¿Y si alguien manda un número en lugar de un nombre?

JSX/TSX: ¡Puedo ayudar a detectar ese problema!

COMPONENTES: ¡Entonces eres como un policía del código! ■

JSX/TSX: Exactamente. ■

■ Escena 5: El examen

REACT: Muy bien. Ahora vamos a comprobar si entendieron.

REACT: Primera pregunta: ¿qué es un componente?

COMPONENTES: Una parte reutilizable de una aplicación React.

REACT: ¡Correcto!

REACT: Segunda pregunta: ¿qué es JSX?

JSX/TSX: Una sintaxis que permite escribir una estructura similar a HTML dentro de JavaScript.

REACT: ¡Correcto!

REACT: Última pregunta: ¿qué es TSX?

JSX/TSX: Es TypeScript combinado con JSX y permite trabajar con tipos de datos y detectar errores con mayor facilidad.

REACT: ¡Excelente!

COMPONENTES: ¿Y ahora nos dan recreo?

REACT: Esto es programación, no colegio.

COMPONENTES: Entonces... ¿cuándo llega el café?

JSX/TSX: ¡Yo hago el botón!

COMPONENTES: ¡Yo solo quería tomarlo! ■

■ Final

REACT: Entonces, recordemos:

COMPONENTES: ■ Componentes: son las piezas reutilizables de una aplicación.

JSX/TSX: ■■ JSX: permite combinar JavaScript con una estructura similar a HTML.

JSX/TSX: ■ TSX: combina TypeScript con JSX y permite trabajar con tipos de datos.

REACT: Y juntos nos ayudan a construir interfaces modernas, organizadas y reutilizables.

TODOS: ¡COMPONENTES + JSX + TSX = REACT! ■

COMPONENTES: ¿Y ahora sí... el café?

TODOS: ¡JAJAJA! ■

TODOS: FIN ■

Duración sugerida: 5–7 minutos. La persona que interpreta JSX/TSX cambia de rol verbalmente en la escena 4 para representar primero JSX y luego TSX.